

Prompts:

1) **SCENARIO 03**

Instructions:

- Generate a cart functionaliy Dashboard as a single self-contained HTML file
- [STRICT] No external dependencies — all CSS/JS inline
- [MANDATORY] Sortable/filterable cart table (ID, Title, price, Status of avaiability, discount)
- Responsive layout for desktop and tablet

Context:

shopping apps cart functionality; needs a single-page visual dashboard to review status and trends without a backend.

Example:

Description:to verify cart functionality

Steps: 1) Log in 2) search the product "red colour bluetooth headset" 3) price range 1000 to 20000

Persona:

- Generate: Frontend/QA Tooling Engineer skilled in building HTML/CSS/JS dashboards
- Review: QA Lead, reviewing usability and accuracy of the dashboard

Output Format:

Single .html file — summary on top, chart in the middle, bug table at the bottom.

Tone: [STRICT] Clean, professional dashboard UI copy;

Cart Functionality QA Dashboard

Review shopping-cart scenarios, availability, pricing, discounts, and QA status.

QA REVIEW

TOTAL SCENARIOS
10
Cart checks tracked

OPEN
6
Needs validation

CLOSED
4
Resolved checks

AVAILABLE
6
Items available

DISCOUNTED
8
Discount above 0%

Severity Distribution

Cart-related QA issues grouped by severity.

Severity	Count
P0	1
P1	4
P2	3
P3	2

Cart Verification Flow

Example shopping-cart test path.

1. Log in
Verify authenticated cart access.

2. Search product
"red colour bluetooth headset".

3. Apply price range
Validate products from ₹1,000 to ₹20,000.

4. Add and verify
Check item, price, availability, quantity, and discount.

Cart Scenario Review

Search, filter, and sort cart scenarios.

Search ID, title, price, availability...

All statuses ▾

All availability ▾

All severities ▾

ID ▴	Title ▴	Price ▴	Status of Availability ▴	Discount ▴	Status ▴	Severity ▴
CRT-001	Red Colour Bluetooth Headset	₹2,499	Available	15%	Open	P1
CRT-002	Cart retains item after refresh	₹4,999	Available	10%	Closed	P1
CRT-003	Unavailable product remains addable	₹18,999	Unavailable	20%	Open	P0
CRT-004	Discount not reflected in cart total	₹3,299	Available	25%	Open	P1
CRT-005	Quantity update exceeds stock	₹7,999	Limited	5%	Open	P2
CRT-006	Remove item does not update cart count	₹1,299	Available	0%	Closed	P2
CRT-007	Price range filter mismatch	₹15,999	Available	12%	Closed	P2
CRT-008	Guest cart merges incorrectly after login	₹5,499	Limited	8%	Open	P1
CRT-009	Expired discount still displayed	₹10,999	Available	30%	Open	P3
CRT-010	Empty cart state validation	—	Unavailable	0%	Closed	P3

2) **SCENARIO 04**

Instructions:

- Doninos Pizza app role based access control test case creation
- [STRICT] Functional only — positive and negative
- [MANDATORY] Include TC ID, Priority, Description, Precondition, Steps, Expected/Actual Result, Test Data
- [CRITICAL] Mix of P0, P1, P2
- [MANDATORY] Exactly 8 test cases
- Then switch persona and review, adding comments

Context:

Doninos Pizza apps login feature — testing between the roles(Admin, registered user, guest).

Example:

TC-ID: TC001 Priority: P1

Description: Verify role based access control over the Admin, registered user, guest.

Expected Result: unauthorized access scenarios, login successful, authorized access scenarios

Persona:

- Generate: e comers Domain QA Engineer, expert in functional testing
- Review: Team Lead, reviewing the test cases

Output Format:

8 test cases in a table, followed by a short review summary and Team Lead comments.

Tone:

[STRICT] Professional tone in review comments and summary.

	A	B	C	D	E	F	G
1	TC ID	Prio	Description	Precondition	Steps	Expected / Actual Result	Test Data
1	TC001	P0	Verify Admin can log in and access Admin-authorized functionality	Valid Admin account exists and is active	1. Open Domino's Pizza app. 2. Enter Admin credentials. 3. Click Login. 4. Navigate to Admin functionality.	Expected: Admin login is successful and Admin-authorized functionality is accessible. Actual: To be captured during execution.	Role: Admin Username: admin_test Password: Valid password
2	TC002	P0	Verify Registered User can log in and access registered-user functionality	Valid registered-user account exists	1. Open app. 2. Enter registered-user credentials. 3. Click Login. 4. Navigate to functionality available to registered users.	Expected: Login is successful and registered-user functionality is accessible. Actual: To be captured during execution.	Role: Registered User Username: user_test Password: Valid password
3	TC003	P1	Verify Guest can access functionality permitted without login	User is logged out and no authenticated session exists	1. Open Domino's Pizza app. 2. Continue as Guest, if supported. 3. Browse publicly available functionality. 4. Attempt to access a registered-user-only feature.	Expected: Guest can access permitted public functionality. Registered-user-only functionality requires login or is unavailable. Actual: To be captured during execution.	Role: Guest Authentication: None
4	TC004	P0	Verify Registered User cannot access Admin-only functionality	Registered-user account exists; Admin functionality is restricted by role	1. Log in as Registered User. 2. Attempt to open Admin-only functionality through application navigation. 3. Attempt the same action through a directly accessible application route, if available.	Expected: Access to Admin-only functionality is denied and the user remains within authorized areas. Actual: To be captured during execution.	Role: Registered User Restricted Area: Admin functionality
5	TC005	P0	Verify Guest cannot access authenticated user functionality	User is not logged in	1. Open the app without logging in. 2. Attempt to access a feature requiring authentication, such as account-specific order history functionality. 3. Observe application response.	Expected: Guest access is denied and the app prompts the guest to log in or register where authentication is required. Actual: To be captured during execution.	Role: Guest Authentication: None Restricted Feature: Account-specific functionality
6	TC006	P1	Verify Admin login fails with invalid credentials and Admin functionality remains inaccessible	Admin account exists	1. Open the login screen. 2. Enter a valid Admin username with an invalid password. 3. Click Login. 4. Attempt to access Admin functionality.	Expected: Login fails with an appropriate error message. Admin functionality is not accessible without successful authentication. Actual: To be captured during execution.	Username: admin_test Password: Invalid password
7	TC007	P1	Verify Registered User cannot access Admin functionality after logging out	Registered User has previously logged in; logout functionality is available	1. Log in as Registered User. 2. Verify registered-user functionality is accessible. 3. Log out. 4. Attempt to access authenticated and Admin-only functionality.	Expected: Session is terminated. Previously authorized functionality is no longer accessible without authentication, and Admin functionality remains unavailable. Actual: To be captured during execution.	Role: Registered User Session: Logged out
8	TC008	P2	Verify each role receives the appropriate access when switching between Guest, Registered User, and Admin sessions	Valid Admin and Registered User accounts exist; user can log out	1. Access app as Guest and verify permitted functionality. 2. Log in as Registered User and verify registered-user functionality.	Expected: Each session provides only functionality authorized for its current role. Access from the previous role is not retained after logout/login as another role.	Roles: Guest, Registered User, Admin Accounts: Valid role-specific accounts
	RBAC Test Cases Team Lead Review						

Team Lead Review Summary

The test cases provide **functional positive and negative coverage** for the three requested Domino's Pizza app roles: **Admin, Registered User, and Guest**. The suite covers successful authentication, permitted access, unauthorized access, invalid login, logout/session transition, and role separation. Priorities are distributed across **P0, P1, and P2** based on the functional impact of each scenario.

Team Lead Comments

- **TC001–TC002:** Positive authentication and authorized-access scenarios are appropriately covered for Admin and Registered User.
- **TC003:** Guest access should be validated against the application's defined public functionality.
- **TC004–TC005:** Unauthorized-access scenarios are critical and should be executed against all applicable restricted features.
- **TC006:** Invalid Admin authentication provides negative coverage and confirms that failed authentication does not grant privileged access.
- **TC007:** Logout/session behavior should be verified to ensure previous authorization does not remain available.
- **TC008:** Role switching provides useful end-to-end functional coverage of role isolation.
- **Overall:** Test cases meet the requested **functional-only, positive/negative, 8-case** scope. During execution, the **Actual Result** field should be updated with observed behavior and linked to defects where applicable.



Prompt 3: **SCENARIO 05**

Instructions:

- API test cases for user registration to validate in Postman tool
- [STRICT] Functional only — positive and negative
- [MANDATORY] Include TC ID, Priority, Description, Precondition, Headers, Request Body, Steps, Expected Result, Actual Result, and Test Data
- [CRITICAL] Mix of P0, P1, P2
- [MANDATORY] Exactly 12 test cases
- Then switch persona and review, adding comments

Context:

Postman app's API test cases for user registration feature — using POST method call

Example:

TC-ID: TC001 Priority: P1

Description: Verify API test cases for user registration on POST call, with proper request/response structure, status codes to validate in Postman tool

Expected Result: TC ID, Priority, Description, Precondition, Headers, Request Body, Steps, Expected Result, Actual Result, and response.

Persona:

- Generate: Payroll Domain QA Engineer, expert in functional testing
- Review: Team Lead, reviewing the test cases

Output Format:

12 test cases in a table, followed by a brief review summary and Team Lead comments.

Tone:

[STRICT] Professional tone in review comments and summary.

	A	B	C	D	E	F	
	TC-ID	Priority	Description	Precondition	Headers	Request Body	
1	TC001	P0	Verify successful user registration with all valid mandatory fields.	Registration API is available; test email is not registered.	Content-Type: application/json	{"name":"John Doe","email":"john01@test.com","password":"Test@1234"}	1. Open Postman. 2. Set endpoint. 4. Add headers.
2	TC002	P0	Verify registration fails when a mandatory email field is missing.	API is available.	Content-Type: application/json	{"name":"John Doe","password":"Test@1234"}	1. Configure POST request. 3. Send request.
3	TC003	P0	Verify registration fails when password is missing.	API is available.	Content-Type: application/json	{"name":"John Doe","email":"john02@test.com"}	1. Configure POST request.
4	TC004	P1	Verify registration fails when the email is already registered.	An account already exists with the test email.	Content-Type: application/json	{"name":"John Doe","email":"existing@test.com","password":"Test@1234"}	1. Use an already registered email.
5	TC005	P1	Verify registration fails when an invalid email format is provided.	API is available.	Content-Type: application/json	{"name":"John Doe","email":"john02test.com","password":"Test@1234"}	1. Enter an invalid email format.
6	TC006	P1	Verify registration fails when the password does not meet defined password rules.	API is available; password policy is configured.	Content-Type: application/json	{"name":"John Doe","email":"john03@test.com","password":"123"}	1. Provide a password that does not meet rules. 2. Send request.
7	TC007	P1	Verify registration fails when the request body is empty.	API is available.	Content-Type: application/json	{}	1. Select POST. 2. Send request. 3. Verify response.
8	TC008	P1	Verify registration fails when the request contains malformed JSON.	API is available.	Content-Type: application/json	{"name":"John Doe","email":"john04@test.com","password":"Test@1234"}	1. Enter malformed JSON.
9	TC009	P2	Verify registration handles an invalid/missing Content-Type header.	API is available.	Content-Type: text/plain	{"name":"John Doe","email":"john05@test.com","password":"Test@1234"}	1. Set Content-Type to text/plain. 2. Send request.
10	TC010	P2	Verify registration rejects an invalid value type for a request field.	API is available.	Content-Type: application/json	{"name":12345,"email":"john06@test.com","password":"Test@1234"}	1. Provide a numeric value for name. 2. Send request.
11	TC011	P2	Verify registration rejects a request containing an unsupported/invalid extra field when strict validation is enabled.	API contract defines strict request validation.	Content-Type: application/json	{"name":"John Doe","email":"john07@test.com","password":"Test@1234","extra":"invalid"}	1. Add an unsupported field. 2. Send request.

A1

:

Review Area

	A	B
1	Review Area	Team Lead Comments
2	Overall	The 12 test cases provide functional positive and negative coverage for the user-registration POST API.
3	Priority Coverage	Priority distribution is P0: 3, P1: 5, P2: 4.
4	TC001	Validate required success response fields against the API contract.
5	TC002-TC003	Confirm mandatory-field errors are validated independently.
6	TC004	Verify duplicate-user status and response; avoid creating another account.
7	TC005-TC006	Validate exact email and password rules from the API specification.
8	TC007-TC008	Good negative coverage for empty and malformed payloads.
9	TC009	Confirm the documented status for unsupported Content-Type, such as 415.
10	TC010	Validate field data types against the API schema.
11	TC011	Execute according to the API's strict/lenient schema behavior.
12	TC012	Replace representative values with exact documented boundary limits.
13	All Cases	In Postman, validate HTTP status, response body, error code/message, and user-creation outcome where applicable.